

*Operating  
Systems:  
Internals  
and Design  
Principles*

# Chapter 7 Memory Management

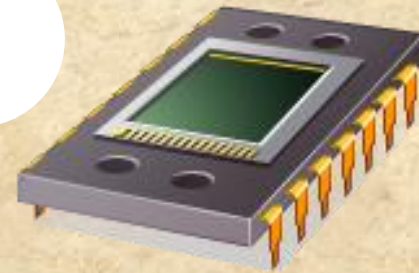
Seventh Edition  
William Stallings

# Memory Management Requirements

- Memory management is intended to satisfy the following requirements:

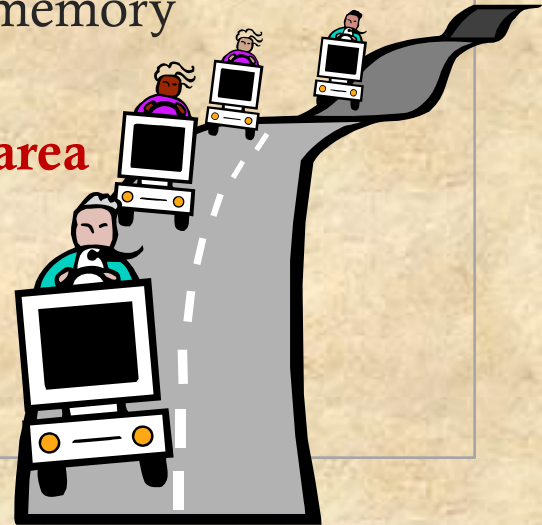
- Relocation
- Protection
- Sharing
- Logical organization
- Physical organization

User and process need the following from memory management



# Relocation

- Programmers typically **do not know in advance which other programs** will be resident in main memory at the time of execution of their program
- Active processes need to be able to be **swapped in and out** of main memory in order to maximize processor utilization
- Specifying that a process must be placed in the same memory region when it is swapped back in would be limiting
  - **may need to *relocate* the process to a different area of memory**

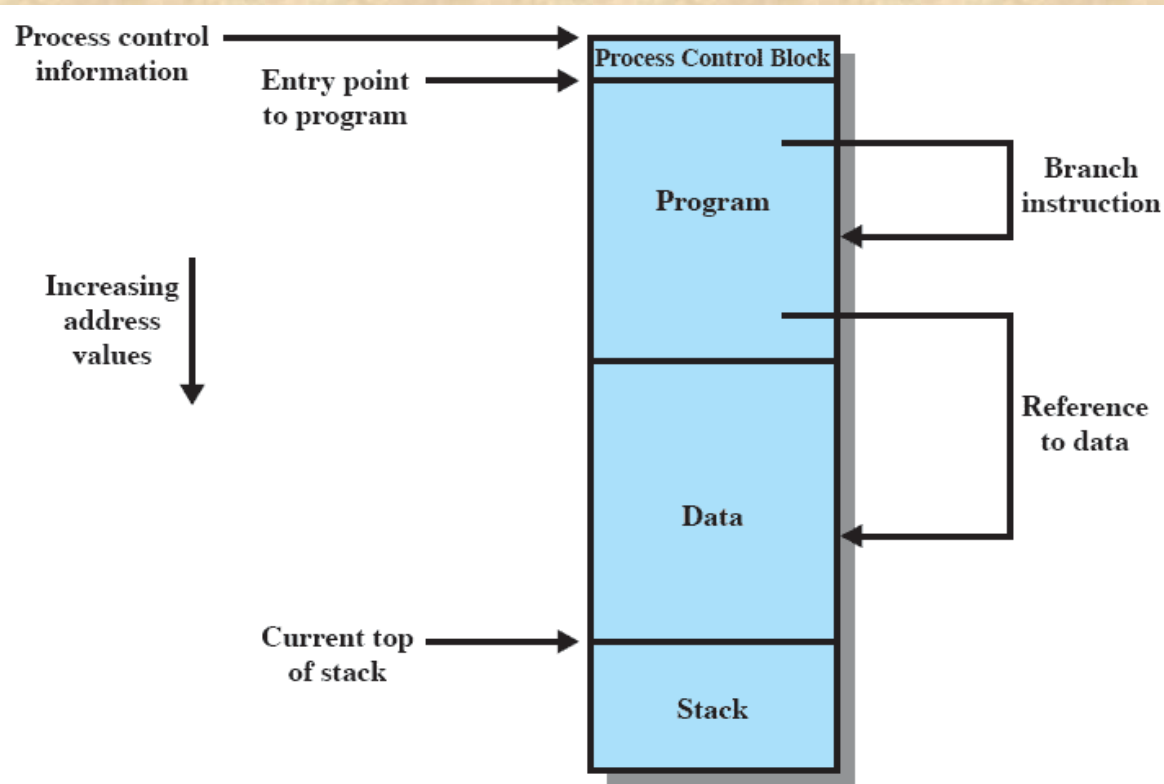




# Protection

- Processes need to acquire permission to reference memory locations for reading or writing purposes
- Location of a program in main memory is unpredictable
- **Memory references generated by a process must be checked at run time**
- **Mechanisms that support relocation also support protection**

# Addressing Requirements



**Figure 7.1** Addressing Requirements for a Process

# Addresses

## Logical

- reference to a memory location independent of the current assignment of data to memory

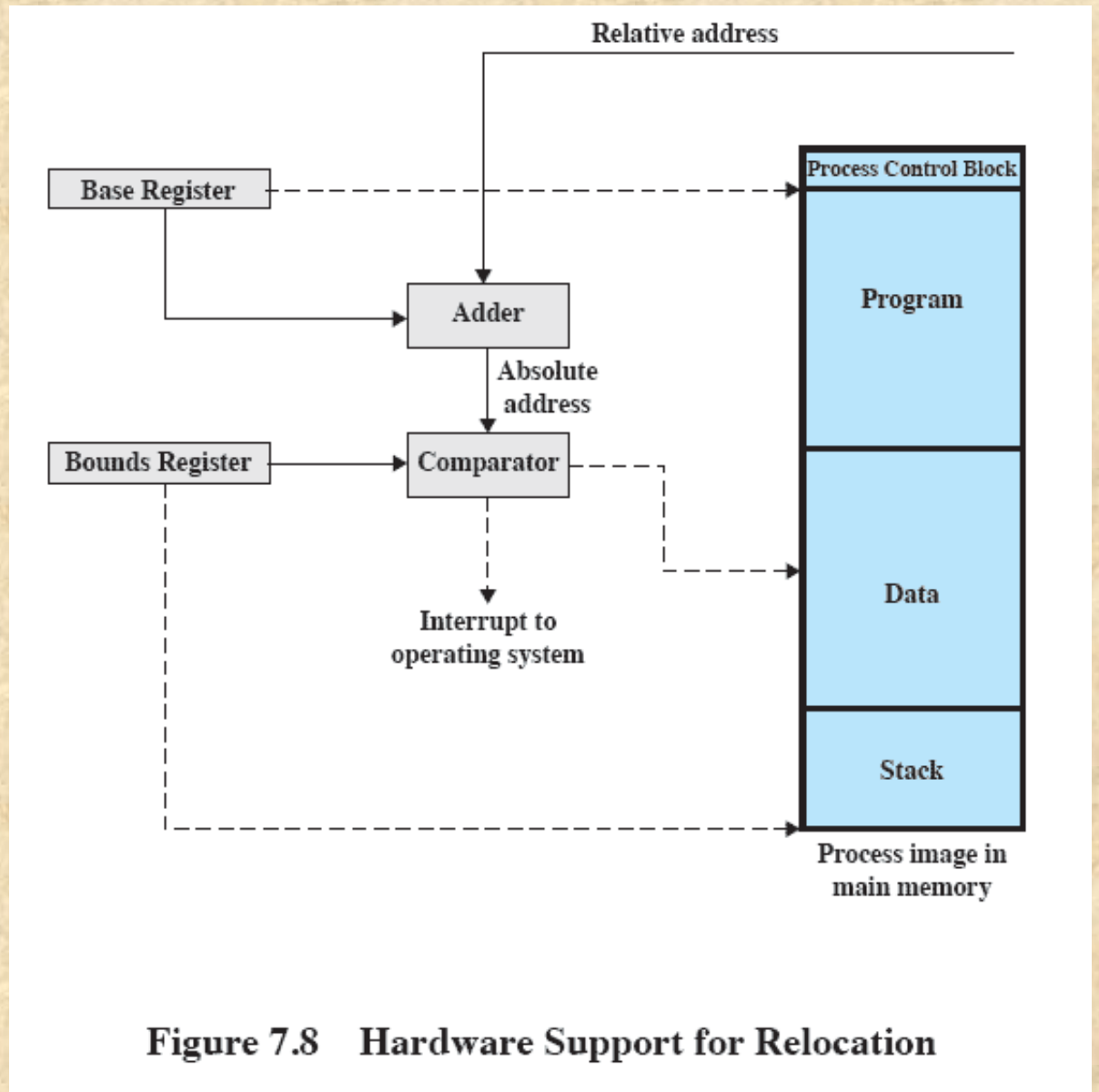
## Relative

- address is expressed as a location relative to some known point

## Physical or Absolute

- actual location in main memory

# Relocation

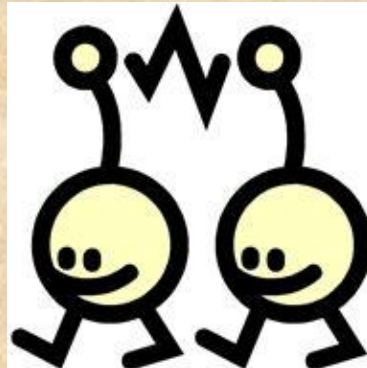


**Figure 7.8 Hardware Support for Relocation**



# Sharing

- Advantageous to allow each process access to the same copy of the program rather than have their own separate copy
- Memory management must allow controlled access to shared areas of memory without compromising protection
- Mechanisms used to support relocation support sharing capabilities





# Logical Organization

- Memory is organized as linear

## Programs are written in modules

- modules can be written and compiled independently
- different degrees of protection given to modules (read-only, execute-only)
- sharing on a module level corresponds to the user's way of viewing the problem

# Physical Organization

Cannot leave the programmer with the responsibility to manage memory

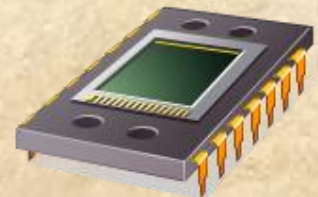
Memory available for a program plus its data may be insufficient

Programmer does not know how much space will be available

*overlaying* allows various modules to be assigned the same region of memory but is time consuming to program

# Memory Partitioning

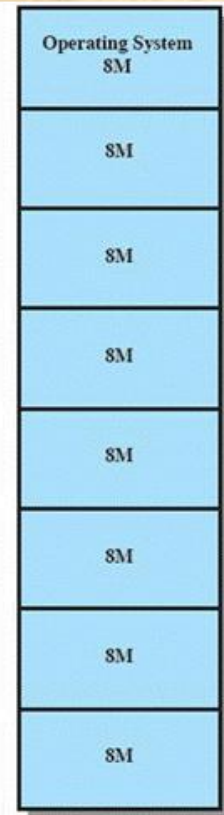
- Memory management brings processes into main memory for execution by the processor
- Partitioning
  - used in several variations in some now-obsolete operating systems





# Fixed Partitioning

- Equal-size partitions
  - any process whose size is less than or equal to the partition size can be loaded into an available partition
- The operating system can swap out a process if all partitions are full and no process is in the Ready or Running state



(a) Equal-size partitions

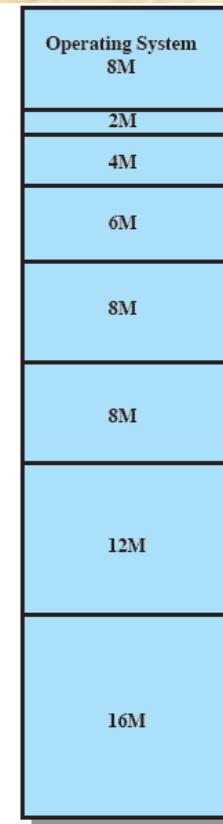
# Disadvantages

- A program may be too big to fit in a partition
  - program needs to be designed with the use of overlays
- Main memory utilization is inefficient
  - any program, regardless of size, occupies an entire partition
  - *internal fragmentation*
    - wasted space due to the block of data loaded being smaller than the partition



# Unequal Size Partitions

- Using unequal size partitions helps lessen the problems
  - programs up to 16M can be accommodated without overlays
  - partitions smaller than 8M allow smaller programs to be accommodated with less internal fragmentation

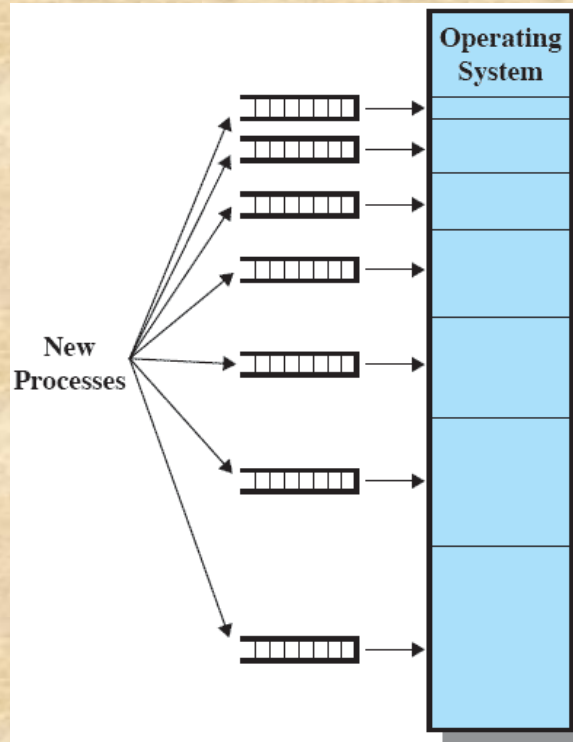


(b) Unequal-size partitions

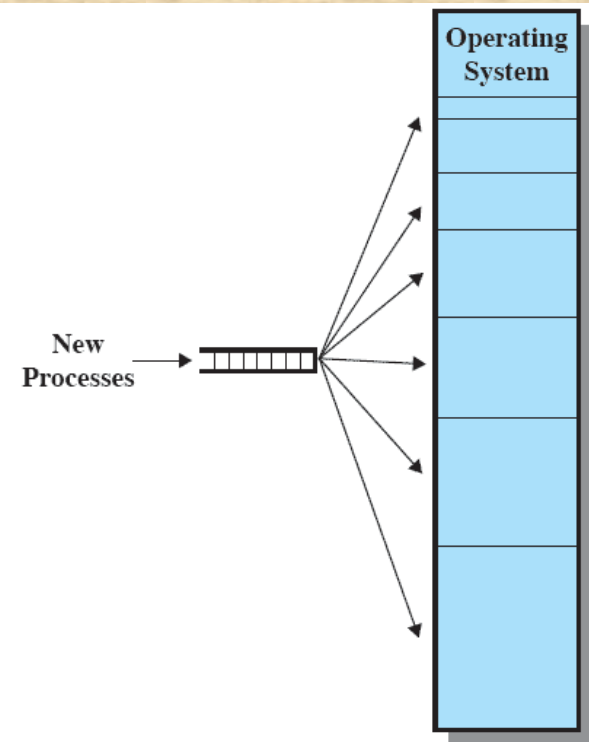


# Memory Assignment

F P n  
i a i  
x r n  
e t g  
d i  
t i  
i  
O



(a) One process queue per partition



(b) Single queue

Figure 7.3 Memory Assignment for Fixed Partitioning

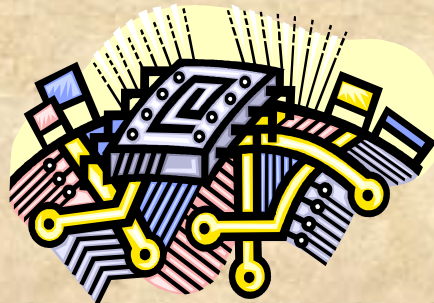
# Disadvantages

- The number of partitions specified at system generation time limits the number of active processes in the system
- Small jobs will not utilize partition space efficiently



# Dynamic Partitioning

- Partitions are of variable length and number
- Process is allocated exactly as much memory as it requires
- This technique was used by IBM's mainframe operating system, OS/MVT





# Effect of Dynamic Partitioning

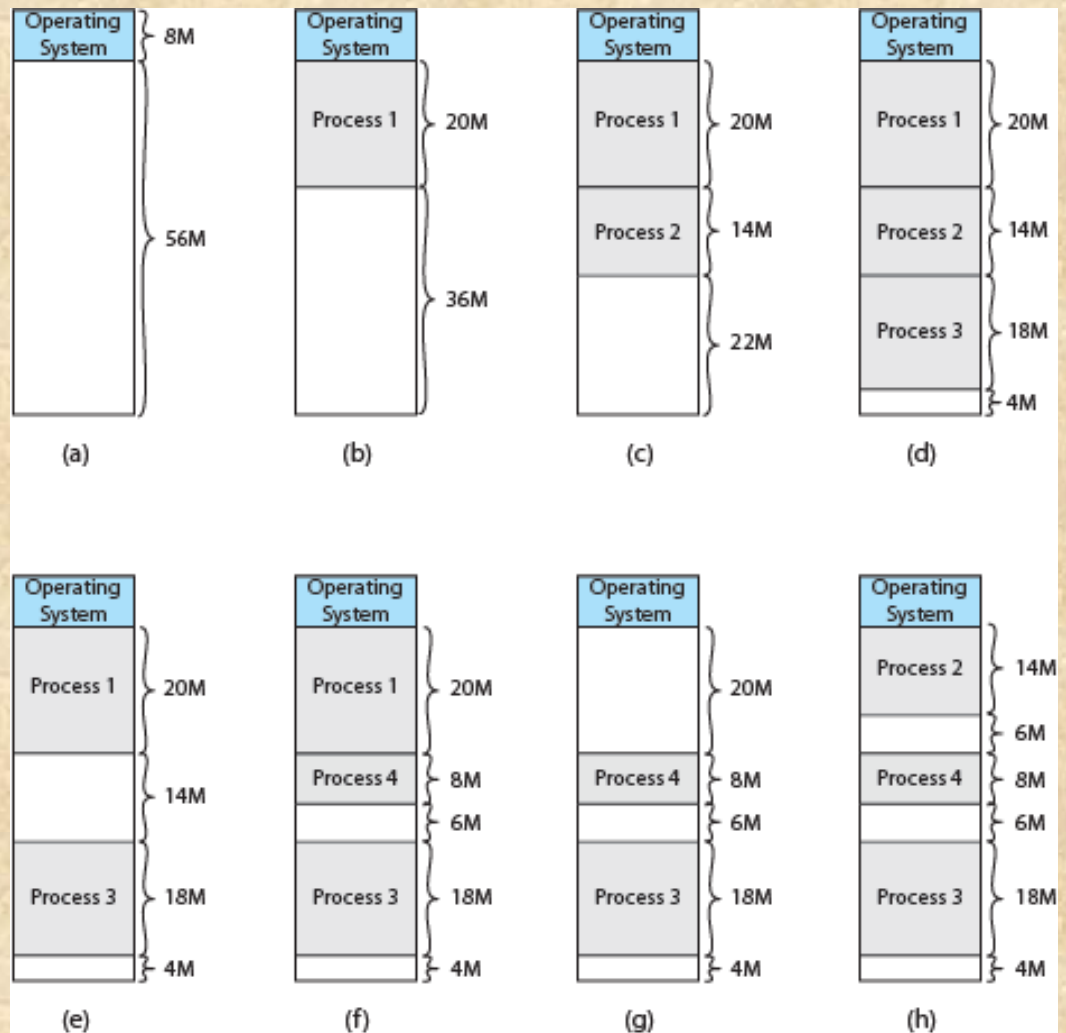


Figure 7.4 The Effect of Dynamic Partitioning

# Dynamic Partitioning

## *External Fragmentation*

- memory becomes more and more fragmented
- memory utilization declines

## *Compaction*

- technique for overcoming external fragmentation
- OS shifts processes so that they are contiguous
- free memory is together in one block
- time consuming and wastes CPU time

# Placement Algorithms

## Best-fit

- chooses the block that is closest in size to the request

## First-fit

- begins to scan memory from the beginning and chooses the first available block that is large enough

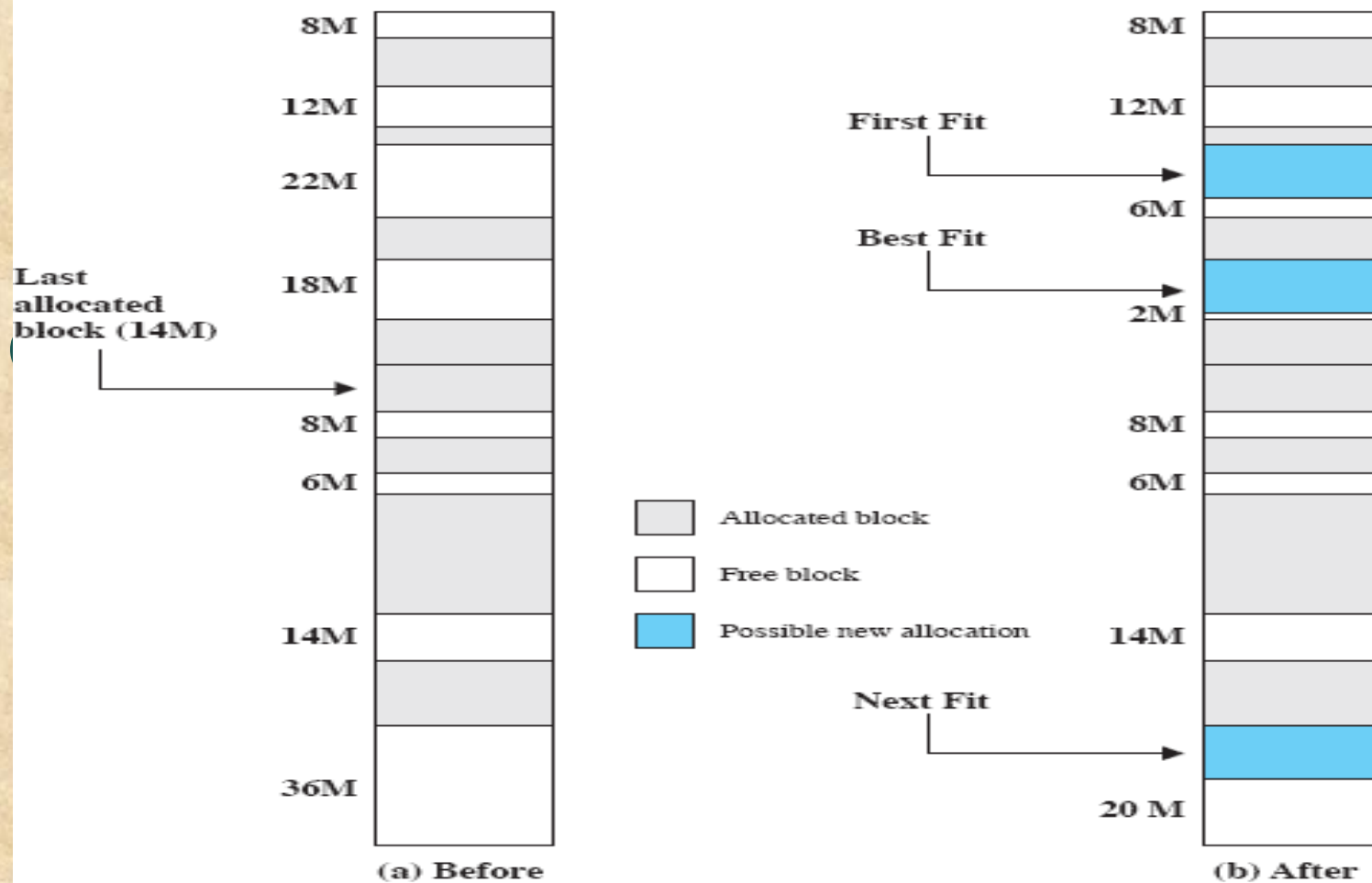
## Next-fit

- begins to scan memory from the location of the last placement and chooses the next available block that is large enough

## worst-fit

- Allocate the *largest* hole; must also search entire list
- Produces the largest leftover hole





**Figure 7.5 Example Memory Configuration before and after Allocation of 16-Mbyte Block**

# Paging

- Partition memory into equal fixed-size chunks that are relatively small
- Process is also divided into small fixed-size chunks of the same size

## *Pages*

- chunks of a process

## *Frames*

- available chunks of memory

| Frame number | Main memory |
|--------------|-------------|
| 0            |             |
| 1            |             |
| 2            |             |
| 3            |             |
| 4            |             |
| 5            |             |
| 6            |             |
| 7            |             |
| 8            |             |
| 9            |             |
| 10           |             |
| 11           |             |
| 12           |             |
| 13           |             |
| 14           |             |

(a) Fifteen Available Frames

| Frame number | Main memory |
|--------------|-------------|
| 0            | A.0         |
| 1            | A.1         |
| 2            | A.2         |
| 3            | A.3         |
| 4            |             |
| 5            |             |
| 6            |             |
| 7            |             |
| 8            |             |
| 9            |             |
| 10           |             |
| 11           |             |
| 12           |             |
| 13           |             |
| 14           |             |

(b) Load Process A

| Frame number | Main memory |
|--------------|-------------|
| 0            | A.0         |
| 1            | A.1         |
| 2            | A.2         |
| 3            | A.3         |
| 4            | B.0         |
| 5            | B.1         |
| 6            | B.2         |
| 7            |             |
| 8            |             |
| 9            |             |
| 10           |             |
| 11           |             |
| 12           |             |
| 13           |             |
| 14           |             |

(c) Load Process B

| Frame number | Main memory |
|--------------|-------------|
| 0            | A.0         |
| 1            | A.1         |
| 2            | A.2         |
| 3            | A.3         |
| 4            | B.0         |
| 5            | B.1         |
| 6            | B.2         |
| 7            | C.0         |
| 8            | C.1         |
| 9            | C.2         |
| 10           | C.3         |
| 11           |             |
| 12           |             |
| 13           |             |
| 14           |             |

(d) Load Process C

| Frame number | Main memory |
|--------------|-------------|
| 0            | A.0         |
| 1            | A.1         |
| 2            | A.2         |
| 3            | A.3         |
| 4            |             |
| 5            |             |
| 6            |             |
| 7            | C.0         |
| 8            | C.1         |
| 9            | C.2         |
| 10           | C.3         |
| 11           |             |
| 12           |             |
| 13           |             |
| 14           |             |

(e) Swap out B

| Frame number | Main memory |
|--------------|-------------|
| 0            | A.0         |
| 1            | A.1         |
| 2            | A.2         |
| 3            | A.3         |
| 4            | D.0         |
| 5            | D.1         |
| 6            | D.2         |
| 7            | C.0         |
| 8            | C.1         |
| 9            | C.2         |
| 10           | C.3         |
| 11           | D.3         |
| 12           | D.4         |
| 13           |             |
| 14           |             |

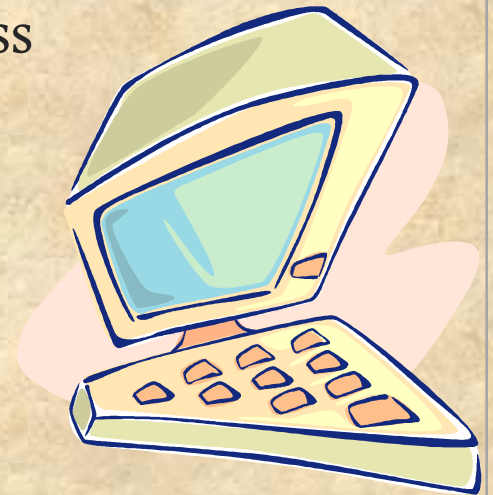
(f) Load Process D

Figure 7.9 Assignment of Process Pages to Free Frames



# Page Table

- Maintained by operating system for each process
- Contains the frame location for each page in the process
- Processor must know how to access for the current process
- Used by processor to produce a physical address



# Data Structures

|   |   |
|---|---|
| 0 | 0 |
| 1 | 1 |
| 2 | 2 |
| 3 | 3 |

Process A  
page table

|   |   |
|---|---|
| 0 | — |
| 1 | — |
| 2 | — |

Process B  
page table

|   |    |
|---|----|
| 0 | 7  |
| 1 | 8  |
| 2 | 9  |
| 3 | 10 |

Process C  
page table

|   |    |
|---|----|
| 0 | 4  |
| 1 | 5  |
| 2 | 6  |
| 3 | 11 |
| 4 | 12 |

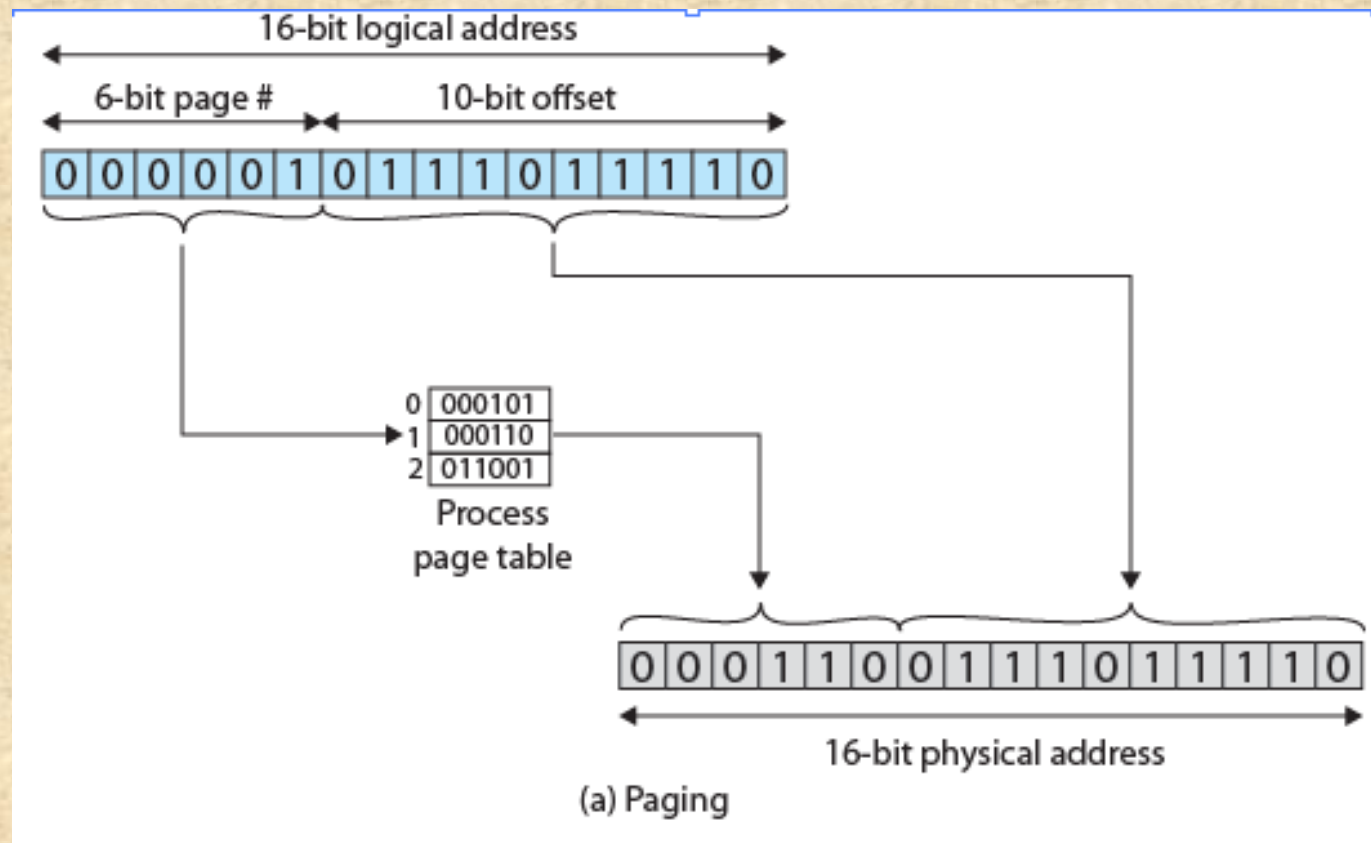
Process D  
page table

|    |
|----|
| 13 |
| 14 |

Free frame  
list

Figure 7.10 Data Structures for the Example of Figure 7.9 at Time Epoch (f)

# Logical-to-Physical Address Translation - Paging





**Q6:** Consider a user program of logical address of size 6 pages and page size is 4 bytes. The physical address contains 300 frames. The user program consists of 22 instructions **a, b, c, . . . u, v** . Each instruction takes 1 byte. Assume at that time the free frames are **7, 26, 52, 20, 55, 6, 18, 21, 70, and 90**.

**Find the following?**

(10 degrees)

- A) Draw the logical and physical maps and page tables?
- B) Allocate each page in the corresponding frame?
- C) Find the physical addresses for the instructions **m, d, v, r**?
- D) Calculate the fragmentation if exist?

|   |                          |
|---|--------------------------|
| 0 | . a<br>. b<br>. c<br>. d |
| 1 | . e<br>. f<br>. g<br>. h |
| 2 | . i<br>. j<br>. k<br>. l |
| 3 | . m<br>. n<br>. o<br>. p |
| 4 | . q<br>. r<br>. s<br>. t |
| 5 | . u<br>. v<br>. .<br>. . |

Logical map

| Page number | Frame number |
|-------------|--------------|
| 0           | 7            |
| 1           | 26           |
| 2           | 52           |
| 3           | 20           |
| 4           | 55           |
| 5           | 6            |

page table

|    |                          |
|----|--------------------------|
| 6  | . u<br>. v<br>. .<br>. . |
| 7  | . a<br>. b<br>. c<br>. d |
|    |                          |
| 20 | . m<br>. n<br>. o<br>. p |
|    |                          |
| 26 | . e<br>. f<br>. g<br>. h |
|    |                          |
| 52 | . i<br>. j<br>. k<br>. l |
|    |                          |
| 55 | . q<br>. r<br>. s<br>. t |

Physical map

**The physical address = page size \* frame number + offset**

The physical address of m =  $4 * 20 + 0 = 80$

The physical address of d =  $4 * 7 + 3 = 31$

The physical address of v =  $4 * 6 + 1 = 25$

The physical address of r =  $4 * 55 + 1 = 221$

The external fragmentation = 0

The internal fragmentation = 2

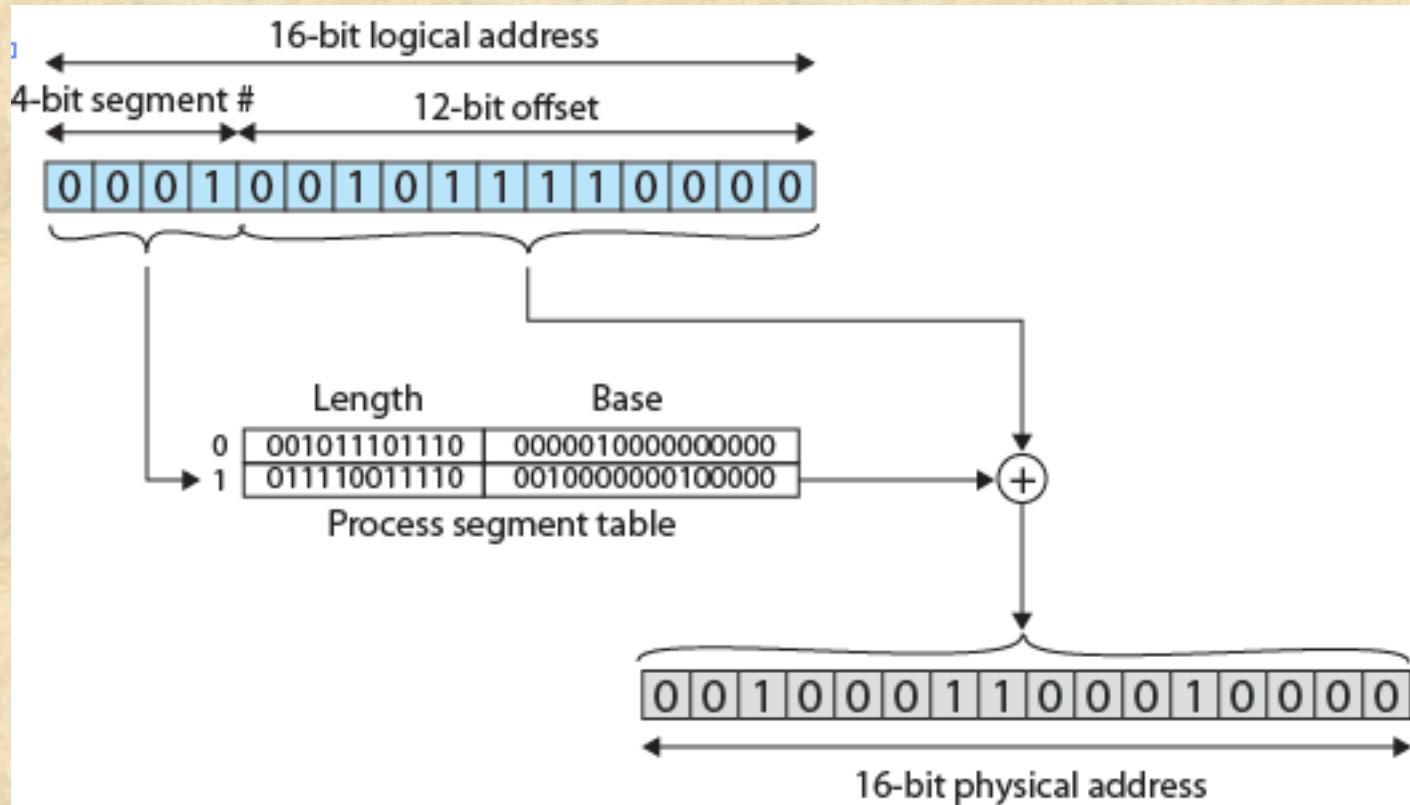


# Segmentation

- A program can be subdivided into segments
  - may vary in length
  - there is a maximum length
- Addressing consists of two parts:
  - segment number
  - an offset
- Similar to dynamic partitioning
- Eliminates internal fragmentation



# Logical-to-Physical Address Translation - Segmentation



# Logical Addresses

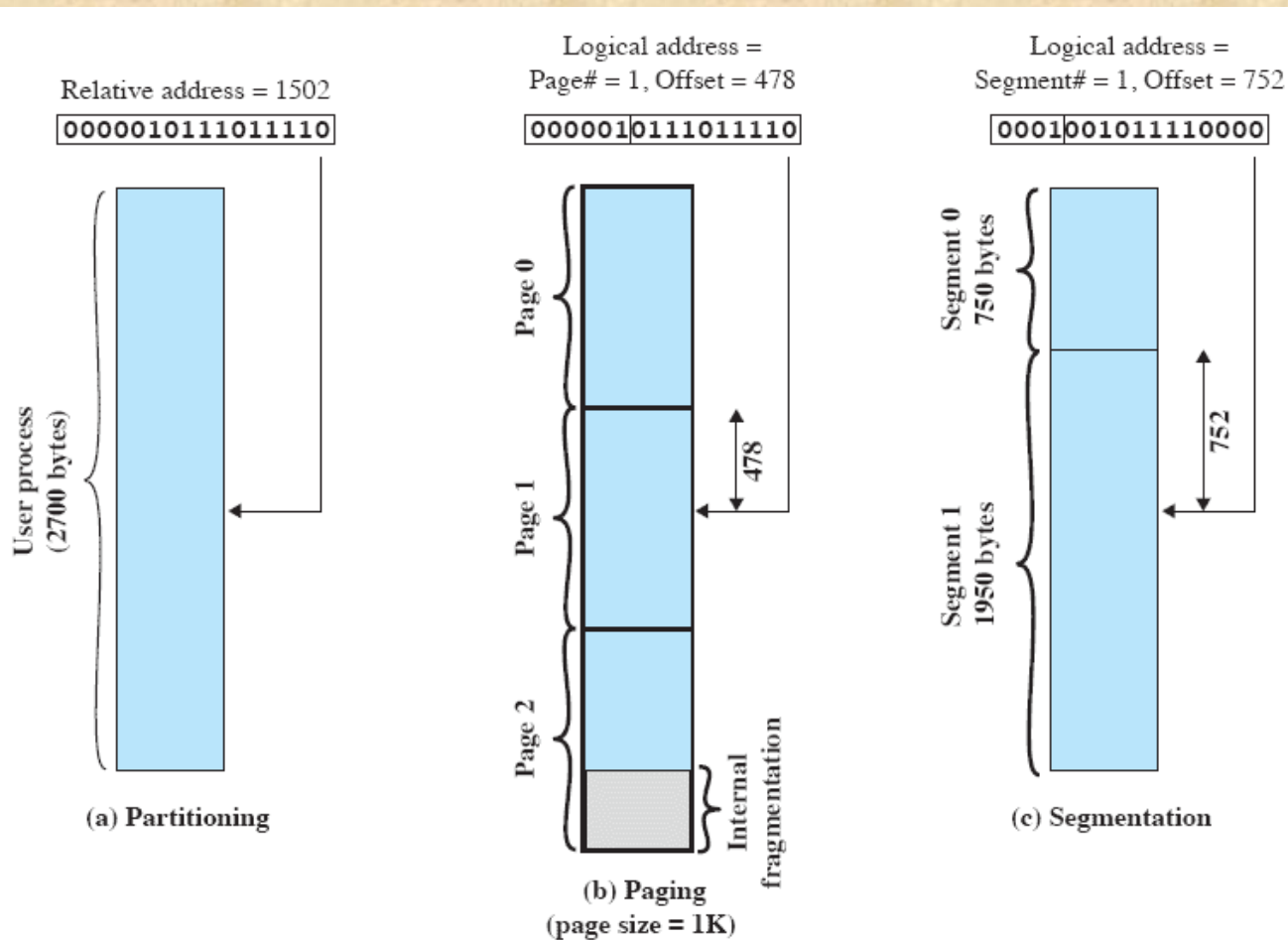


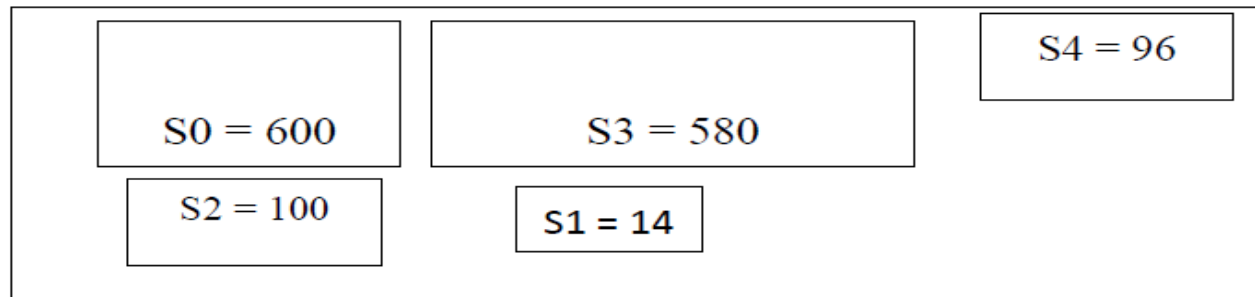
Figure 7.11 Logical Addresses



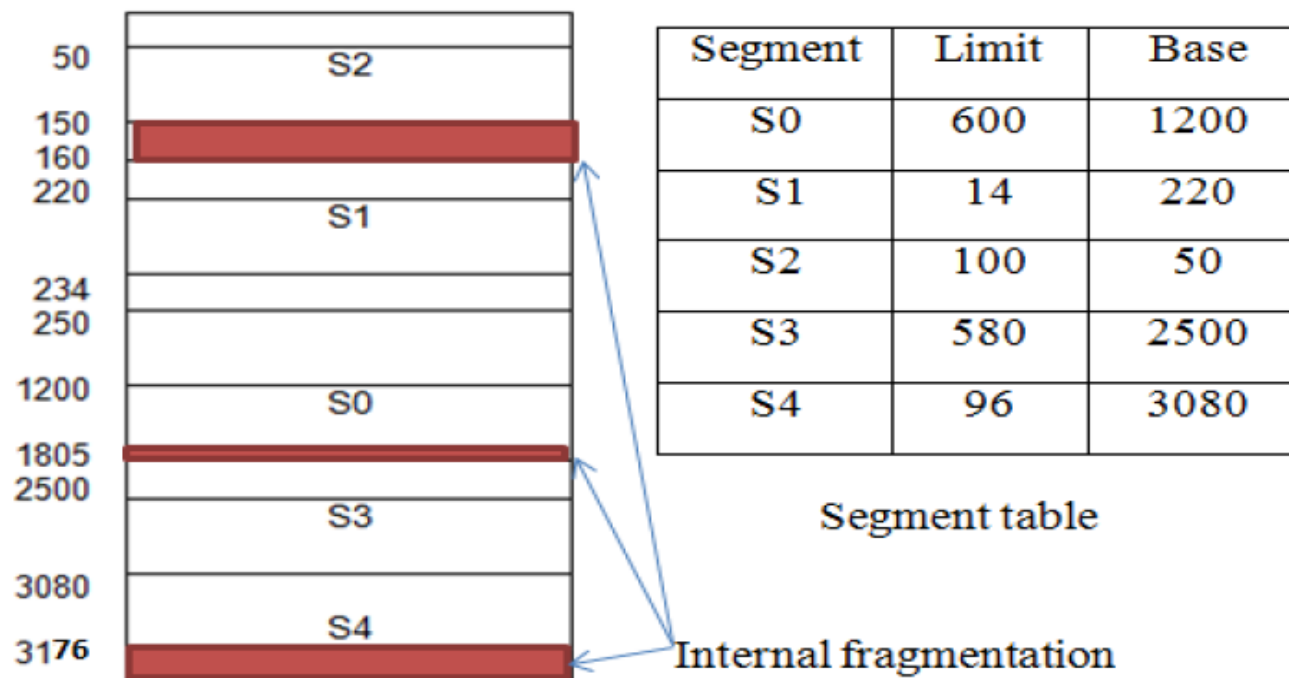
Q7: Consider a program consists of five segments:  $S_0 = 600$ ,  $S_1 = 14$  KB,  $S_2 = 100$  KB,  $S_3 = 580$  KB, and  $S_4 = 96$  KB. Assume at that time, the available free

space partitions of memory are 1200–1805, 50 – 150, 220-234, and 2500-3180.  
Find the following:

1. Draw logical to physical maps and segment table?
2. Allocate space for each segment in memory?
3. Calculate the external fragmentation and the internal fragmentation?
4. What are the addresses in physical memory for the following logical addresses: 0.580, (b) 1.17 (c) 2.66 (d) 3.82 (e) 4.20?



Logical map



Segment table

Internal fragmentation

Physical memory

External fragmentation = 0.

$$\begin{aligned}\text{Internal fragmentation} &= (160-150) + (1805-1800) + (3180-3176) \\ &= 10 + 5 + 4 = 19\end{aligned}$$

$$\text{Fragmentation} = \text{external fragmentation} + \text{internal fragmentation} = 0 + 19 = 19$$

4. The physical addresses are

- a) 0.580 → the physical address of 0.580 = 1200 + 580 = 1780.
- b) 1.17 → because  $d > \text{limit of S1}$ , the address is **wrong**.
- c) 2.66 → the physical address of 2.66 = 50 + 66 = 116
- d) 3.82 → the physical address is of 3.82 is = 2500 + 82 = 2582
- e) 4.20 → the physical address 4.20 = 3080 + 20 = 3100



# Summary

## ■ Memory Management

- one of the most important and complex tasks of an operating system
- needs to be treated as a resource to be allocated to and shared among a number of active processes
- desirable to maintain as many processes in main memory as possible
- desirable to free programmers from size restriction in program development
- basic tools are paging and segmentation (possible to combine)
  - paging – small fixed-sized pages
  - segmentation – pieces of varying size